NAZARBAYEV UNIVERSITY

Programming for Engineers

Week 13: Python and Programming Beyond Python

Yanwei Wang
School of Engineering
Nazarbayev University

Fall 2026



Week 13 at a glance

Lecture 37

Python and transferable ideas

Engineering workflows, language choices, runtime models, and translation contracts.

Lecture 38

MATLAB for Python users

Indexing, arrays, operators, functions, plots, and a cooling-model translation.

Lecture 39

C++ and Java

Read familiar logic in typed programs; preserve units, boundaries, and object behavior.

Keep the problem and evidence.
Change the language deliberately.

[C] Session 13, CLOs 1, 2, 3, 5; [W] Sections 16.7–16.13.

Scope: transfer, not three new courses

Official Session 13

Python usage and other programming languages: Java, C+, MATLAB, etc.
The outline's **C+ is interpreted here as C++**; C is a separate optional comparison.

Core expectation

Read, explain, modify, and test small familiar calculations in another syntax.

Not required here

New development stacks, advanced pointers, templates, GUI frameworks, or a new assessment.

[C] p. 5, Sessions 13–14. Java/C++ examples are original additions; Week 14 is course review.

NAZARBAYEV UNIVERSITY

LECTURE 37

Python in Engineering and Transferable Ideas

Use a familiar problem to learn an unfamiliar language

NAZARBAYEV UNIVERSITY

Lecture 37 learning goals

1. Explain Python's role in a small engineering workflow.
2. Distinguish a language, a library, an editor, and a runtime.
3. Recognize concepts that transfer across languages.
4. Identify numeric and syntactic translation traps.
5. Specify tests before accepting a translated program.

[W] Sections 16.7–16.13; original teaching adaptation.

NAZARBAYEV UNIVERSITY

Warm-up: what must stay the same?

```
# Familiar example from the course book:
    force in N, area in m^2.
force_n = 5000.0
area_m2 = 0.002
stress_pa = force_n / area_m2
# Convert only for presentation; keep the
    unit visible.
print(f"Stress = {stress_pa / 1e6:.2f} MPa")
```

Individual, then pairs - 3 min

Before running

1. Name the input and output units.
2. Calculate the answer by hand.
3. Identify what another language must preserve.

[W] Section 16.7.1; output formatting added for explicit units.

NAZARBAYEV UNIVERSITY

You are not starting from zero

Already learned	Transferable question
Variables and types	What quantity or record does this value represent?
Conditions and loops	Which rule selects the action? What repeats?
Functions and classes	What is the interface and responsibility?
Testing and debugging	What independently justified result should occur?
Efficiency and search	How does the work depend on the input?

The durable skill

Translate the model, the algorithm, and the evidence—not just punctuation.

[W] Sections 16.11–16.12.

NAZARBAYEV UNIVERSITY

Language, library, editor, runtime

Term	Meaning here	Example
Language	Rules for expressing computations	Python; C++; Java
Library	Reusable implementations and interfaces	NumPy; Matplotlib
Editor / IDE	A place to write and inspect source	VS Code
Runtime / toolchain	Tools that execute or build the program	Python interpreter; JDK; compiler

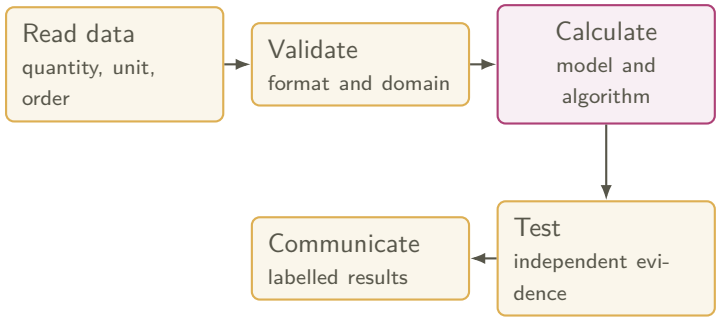
Avoid a common mix-up

Installing an editor is not the same as installing the language tools.

[P] Python Tutorial; [J1] Getting Started with Java; teaching definitions.

NAZARBAYEV UNIVERSITY

A small engineering workflow



Python can coordinate the stages

The algorithm is only one part of a reproducible computational result.

[W] Sections 1.4, 16.5, 16.11.

NAZARBAYEV UNIVERSITY

Python example: preserve the warning locations

```python
# A complete small workflow; the numbers are synthetic teaching data.
readings_c = [19.5, 20.0, 25.0, 30.0, 30.5]
low_c, high_c = 20.0, 30.0
# Preserve the input order and the position of each warning.
warnings = []
for index, value_c in enumerate(readings_c):
    if value_c < low_c or value_c > high_c:
        warnings.append((index, value_c))
# A mean summarizes values, but cannot replace individual warnings.
mean_c = sum(readings_c) / len(readings_c)
print(f"Mean = {mean_c:.2f} deg C")
print("Warnings:", warnings)
print("Raw readings:", readings_c)
```

[A] Original small workflow using Weeks 2, 5, and 7.

NAZARBAYEV UNIVERSITY

A summary is not the whole result

```
Mean = 25.00 deg C
Warnings: [(0, 19.5), (4, 30.5)]
Raw readings: [19.5, 20.0, 25.0, 30.0, 30.5]
```

Inspect three kinds of evidence

- Mean: one aggregate number.
- Warnings: values and original indices.
- Raw list: acquisition order retained.

Interpretation

A mean of $25.0\,^{\circ}\mathrm{C}$ does not imply that every reading satisfies the limits.

[A] Captured Python output; hand-checked against the five supplied values.

NAZARBAYEV UNIVERSITY

Python tools occupy different layers

Layer	Examples	Purpose in this course
Built-in constructs	lists, loops, functions	Describe data and algorithms
Standard library	math, csv, pathlib	Common supporting tasks
Third-party libraries	NumPy, Matplotlib	Numeric arrays and graphics
Your own code	checked sensor functions	The problem-specific rules

This week

NumPy appears only where an array comparison helps. Most Python examples use the standard library.

[W] Sections 16.5.3 and 16.9; [P] Python Tutorial.

NAZARBAYEV UNIVERSITY

Read an unfamiliar call through its contract

```
# Function name alone does not explain all of its behavior.
import math
root = math.sqrt(9.0)
print(root)
# Read documentation before using a new domain or shape.
help(math.sqrt)
```

Look for

- Accepted input types and valid domain.
- Returned type, shape, units, and failure behavior.
- A small example that you can verify independently.

[W] Section 16.12; [P] Python Tutorial and standard library.

NAZARBAYEV UNIVERSITY

Same formula, different surface syntax

Python

```
# V and ohm are the chosen units.
voltage_v = 12.0
resistance_ohm = 4.0
current_a = voltage_v / resistance_ohm
print(current_a)
```

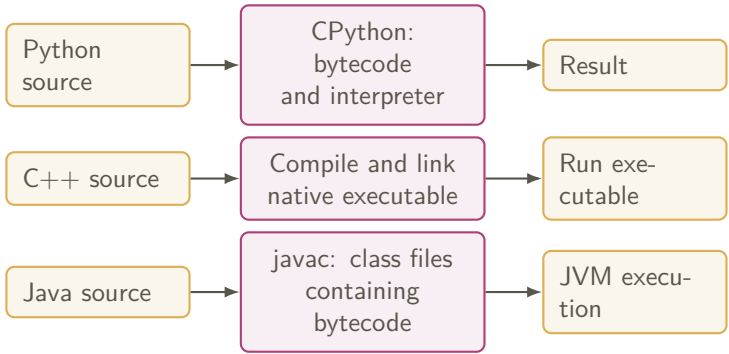
MATLAB

```
% Same quantities and scalar division.
voltage_v = 12.0;
resistance_ohm = 4.0;
current_a = voltage_v / resistance_ohm;
disp(current_a)
```

[W] Sections 16.7.1–16.7.3.

NAZARBAYEV UNIVERSITY

Execution involves more than an editor



[P] Tutorial, compiled Python files; [J1] Java execution cycle; [A] C++ toolchain model.

NAZARBAYEV UNIVERSITY

Dynamic and static typing: a first comparison

Python

```
# A name can later refer to another type.
value = 3
value = "three"
print(type(value).__name__)
```

C++ or Java

```
// This variable is declared as an integer.
int value = 3;
// value = "three"; // Not a valid assignment
```

Neither model removes testing

Static checks can reject some type mismatches; they cannot infer your engineering intention.

[P] Python Tutorial; [J2] Primitive types; [A] introductory C++ comparison.

NAZARBAYEV UNIVERSITY

A valid numeric type does not enforce units

```
// Valid C++ and Java declarations, but a misleading unit label.
double forceN = 10000.0;
double areaMm2 = 500.0;
double stressPa = forceN / areaMm2; // 20 means N/mm^2, not Pa.
```

Repair the meaning

Either convert area to m^2 before calculating pascals, or report $20\,\mathrm{MPa}$ because $1\,\mathrm{N/mm^2} = 1\,\mathrm{MPa}$.

[W] Section 16.6.1; original typed-language adaptation.

NAZARBAYEV UNIVERSITY

Recognize the syntax that changes

Idea	Python	MATLAB	C++ / Java
Comment	`# ...`	`% ...`	`// ...`
Statement ending	usually newline	`;` suppresses echo	`;`
Block	colon + indentation	`end`	`{ ... }`
Equality test	`==`	`==`	`==` for numbers
Not equal	`!=`	`~=`	`!=`

Always inspect context

Similar punctuation can have different meanings for arrays, strings, or objects.

[W] Sections 16.7 and I.8; [J2] Operators; [M2] MATLAB operations.

NAZARBAYEV UNIVERSITY

Trap 1: division depends on operand types

Expression	Python	MATLAB doubles	C++ / Java
`7 / 2`	3.5	3.5	3
`7.0 / 2.0`	3.5	3.5	3.5

```
// A floating destination cannot restore a fraction already discarded.
double lost = 7 / 2;       // 3.0
double kept = 7.0 / 2.0;   // 3.5
```

Predict before changing the operands

[C1] C++ draft, expr.mul; [J3] JLS 15.17.2; [P] numeric tutorial.

NAZARBAYEV UNIVERSITY

Trap 2: a caret is not always a power

Required operation	Python	MATLAB	C++ / Java
Square the scalar x	`x ** 2`	`x ^ 2`	`x * x`
Evaluate 3 squared	9	9	9 with multiplication
Evaluate literal `3 ^ 2`	1 (XOR)	9 (power)	1 (XOR)

Do not translate symbol by symbol

In MATLAB, array elementwise powers use `.^`. In C++ and Java, `^` is not exponentiation.

[M2] MATLAB array/matrix operators; [J3] JLS 15.22.1; [C2] C++ expr.xor.

NAZARBAYEV UNIVERSITY

Translate a contract before translating code

Part of the promise	Example
Inputs and units	Force in N; area in mm^2
Valid domain	Finite real scalars; area strictly positive
Output	Uniform axial stress in MPa
Failure behavior	Reject invalid area; do not invent a stress
Evidence	10000 N and 500 mm^2 give 20 MPa

Boundary tests travel with the algorithm

Keep zero force, negative force under the signed convention, and zero-area rejection.

[W] Sections 16.6, 16.11; [A] shared cross-language interface.

NAZARBAYEV UNIVERSITY

Compare results and behavior, not just text

Check	Expected	Python / C++ / Java
Stress: F=10000, A=500	20 MPa	20.000000 in all three
Readings strictly above 30	1	1 in all three
Mean of the five readings	25 deg C	25.000000 in all three
Reject update to -1	rejected	rejected in all three
State after rejected update	12	12.000000 in all three

Evidence status

These outputs were executed. MATLAB examples have source review and supplied native checks, not a claimed local MATLAB run.

[A] data/cross_language_checks.csv; tests/verify_all.py.

NAZARBAYEV UNIVERSITY

Reproducibility includes the toolchain

Record what actually ran

- Source files and input data.
- Compiler or interpreter version.
- Libraries, flags, and run command.

Do not infer from the language name

- A faster language cannot fix a wrong formula.
- The same syntax can call different library work.
- A screenshot does not replace runnable files.

[W] Sections 16.5 and 16.12; [A] package verification record.

NAZARBAYEV UNIVERSITY

Choose a tool from the constraints

1. What must the program connect to or run on?
2. Which libraries already solve the required task?
3. What performance, accuracy, and memory evidence is needed?
4. Who must maintain, inspect, and reproduce the program?

A useful decision

The strongest choice is a justified fit to the problem, not the longest list of languages.

[W] Sections 16.9–16.10.

NAZARBAYEV UNIVERSITY

Activity: defend a tool choice

Hypothetical task	What should the team investigate?
Batch-process sensor CSV files	File interfaces, validation, reproducible reports
Extend an existing controls model	Installed environment and compatible tool-boxes
Deploy on a constrained device	Target hardware, compiler, memory, timing
Maintain an existing Java service	Existing interfaces, build tools, team tests

Pairs - choose one task and justify two constraints - 4 min

[W] Section 16.10; Java scenario is an original extension for the course scope.

NAZARBAYEV UNIVERSITY

AI-assisted translation still needs independent checks

A learning-focused request

Translate this small function while preserving its units, valid inputs, strict/inclusive boundaries, returned result, and failure behavior. Explain every changed construct.

Before retaining the result

- Read the official interface documentation.
- Run a known answer and a boundary case.

Keep responsibility

- Do not share confidential data.
- Follow the current task's assistance rules.

[W] Section 16.13 and Appendix M.

NAZARBAYEV UNIVERSITY

Learn a new language through a familiar problem

1. Run the smallest valid program.
2. Translate one formula and label the result.
3. Add one condition, loop, and function.
4. Learn the main collection and its indexing rules.
5. Rebuild one known example and reuse its tests.

Then consult deeper references

Only add features when the problem calls for them.

[W] Section 16.12.

Lecture 37 exit ticket

Explain without running

1. Why can `double x = 7 / 2;` lose a fraction?
2. Which parts of a stress-calculation contract must not change?
3. Why is agreement between two programs not a complete validation?

Individual - 3 min

Next lecture

Use the same transfer method for MATLAB arrays, plots, and a small simulation.

[A] Synthesis of Lecture 37.

NAZARBAYEV UNIVERSITY

LECTURE 38

MATLAB for Python Users

Preserve the algorithm while learning array-oriented syntax

NAZARBAYEV UNIVERSITY

Lecture 38 learning goals

1. Read MATLAB scalar calculations and formatted output.
2. Translate indexing, ranges, and array operations deliberately.
3. Recognize conditions, loops, and function interfaces.
4. Compare a Python and MATLAB cooling update.
5. Check numerical results and the meaning of every axis.

[W] Sections 16.7–16.8; selected Appendix I.

NAZARBAYEV UNIVERSITY

Warm-up: what kind of object is this?

```python
# An ordinary Python list repeats.
values = [1, 2, 3]
print(values * 2)
```

```python
# A NumPy array performs numeric operations.
import numpy as np
values = np.array([1.0, 2.0, 3.0])
print(values * 2)
```

Predict

The outputs are not equivalent because the types are not equivalent.

Pairs - 2 min

[W] Sections 16.7.4–16.7.6; [N] NumPy fundamentals.

NAZARBAYEV UNIVERSITY

Scalar calculations: familiar reasoning

```
% Same force and area as the Python version; no toolboxes are needed.
force_n = 5000.0;     % N
area_m2 = 0.002;      % m^2
stress_pa = force_n / area_m2;
% A semicolon suppresses echo; fprintf still prints intentionally.
fprintf('Stress = %.2f MPa\n', stress_pa / 1e6);
```

Read the changes

- % begins a comment.
- ; suppresses the automatic echo.
- `fprintf` formats an intentional report.

[W] Sections 16.7.1–16.7.3.

NAZARBAYEV UNIVERSITY

An array has a shape as well as values

```
% One row, three columns.
row = [1 2 3];
% Three rows, one column.
column = [1; 2; 3];
disp(size(row));
disp(size(column));
```

Expected dimensions

Object	Shape
row	1 by 3
column	3 by 1

Keep engineering meaning

A row and a column are not interchangeable merely because both contain three numbers.

[W] Appendix I.6; [M1] MATLAB array indexing and arrays.

NAZARBAYEV UNIVERSITY

Translate positions, not written index numbers

Meaning	Python sequence	MATLAB vector
First element	`x[0]`	`x(1)`
Last element	`x[-1]`	`x(end)`
Second through fourth	`x[1:4]`	`x(2:4)`
Number of elements	`len(x)` for a list	`numel(x)`

Two different endpoint rules

Python slice stops are excluded. MATLAB colon indices include the endpoint when the progression reaches it.

[W] Section 16.7.4; Appendix I.7; [M1], [M3].

NAZARBAYEV UNIVERSITY

Activity: select the same three measurements

```
# Python baseline: the values are 20, 30, 40.
x = [10, 20, 30, 40, 50]
selected = x[1:4]
```

Proposed MATLAB A

```
% Does this select the same values?
selected = x(1:4);
```

Proposed MATLAB B

```
% Does this select the same values?
selected = x(2:4);
```

Predict values, not just lengths - 3 min

[W] Appendix I.7.1–I.7.2; original exercise.

NAZARBAYEV UNIVERSITY

Create a grid with a stated endpoint policy

Goal	Python	MATLAB
Integer samples 0 to 5	`range(0, 6)`	`0:5`
101 points including 0 and 10	`np.linspace(0,10,101)`	`linspace(0,10,101)`
Step 2 starting at 0 up to 5	`range(0,6,2)`	`0:2:5`

Inspect the result

The last row yields 0, 2, 4 in both cases; the upper bound is not reached. Floating grids need an explicit count or tolerance.

[W] Section 16.7.5; [M3] colon; [N] linspace.

NAZARBAYEV UNIVERSITY

Elementwise division pairs the measurements

```
% Each area belongs to the force at the same position.
force_n = [1000.0 2000.0 3000.0];
area_mm2 = [100.0 100.0 100.0];
stress_mpa = force_n ./ area_mm2;
disp(stress_mpa);  % Expected: 10 20 30.
```

Why the dot matters

./ divides corresponding elements. Plain / on nonscalar arrays asks a different matrix question.

[W] Section 16.7.6 and Appendix I.8; [M2] array vs. matrix operations.

NAZARBAYEV UNIVERSITY

Matrix power and elementwise power differ

```
% Use this small matrix to check by hand.
A = [1 2; 3 4];
entry_squares = A .^ 2;
matrix_square = A ^ 2;
```

Different questions

$$\texttt{A .\^{} 2} = \begin{bmatrix} 1 & 4 \\ 9 & 16 \end{bmatrix}$$

$$A^2 = \begin{bmatrix} 7 & 10 \\ 15 & 22 \end{bmatrix}$$

Python with NumPy

`A ** 2` is elementwise; `A @ A` is matrix multiplication.

[W] Section 16.7.6; [M2]; [N] array operations.

NAZARBAYEV UNIVERSITY

Logical masks retain matching values

```
% The limits are inclusive, as in the earlier Python example.
temperature_c = [19.5 20.0 25.0 30.0 30.5];
mask = temperature_c >= 20.0 & temperature_c <= 30.0;
within_c = temperature_c(mask);
disp(within_c);          % Expected: 20 25 30.
fprintf('%d\n', sum(mask));  % Three true entries.
```

Scalar or array question?

Use && and || for short-circuit scalar logic. Use & and | to combine elementwise masks.

[W] Appendix I.7.4 and I.8.5; [M2].

NAZARBAYEV UNIVERSITY

Translate text comparisons deliberately

```python
# Python compares string content.
unit = "kPa"
print(unit == "kPa")
```

```matlab
% Here unit is a MATLAB character vector.
unit = 'kPa';
matched = strcmp(unit, 'kPa');
fprintf('%d\n', matched);
```

Keep this example simple

MATLAB also has string arrays. Do not mix character-vector and string-array assumptions without checking the interface.

[W] Appendix I.9; [M4] strcmp.

NAZARBAYEV UNIVERSITY

Conditionals preserve the boundary rule

```python
# The exact boundary passes.
fos = 2.0
if fos >= 2.0:
    status = "within stated rule"
else:
    status = "below stated rule"
```

```
% Same comparison; different block ending.
fos = 2.0;
if fos >= 2.0
    status = 'within stated rule';
else
    status = 'below stated rule';
end
```

One symbol changes the specification

Replacing >= with > is a behavioral change, not merely a new language style.

[W] Section 16.7.8; Week 7 boundary-test connection.

NAZARBAYEV UNIVERSITY

A loop still repeats the same calculation

```python
# Sample t = 0, 1, ..., 5 seconds.
for time_s in range(0, 6):
    velocity_m_s = 9.81 * time_s
    print(time_s, velocity_m_s)
```

```
% The same six sample times.
for time_s = 0:5
    velocity_m_s = 9.81 * time_s;
    fprintf('%d %.2f\n', ...
        time_s, velocity_m_s);
end
```

The loop variable is scalar here

Use ordinary scalar arithmetic inside this loop. Array dots are needed when the operands represent elementwise array work.

[W] Sections 16.7.7–16.7.9; adapted constant-acceleration example.

NAZARBAYEV UNIVERSITY

Functions expose inputs and an output name

```python
def count_above(values, limit):
    """Count values strictly above a limit;
     assume finite numbers."""
    count = 0
    for value in values:
        if value > limit:  # Equality is not
     a warning.
            count += 1
    return count
```

```
function count = count_above(values, limit)
% Count values strictly above limit; assume a
      finite numeric vector.
count = 0;
for i = 1:numel(values)
    if values(i) > limit   % Equality is
     deliberately excluded.
        count = count + 1;
    end
end
end
```

MATLAB file convention

Save the primary function in `count_above.m`; call `count_above(values, 30)` from the same folder or search path.

[W] Section 16.7.10; [M5] function declaration; original matched count example.

NAZARBAYEV UNIVERSITY

Tests can be translated before the main program

```
% Run beside count_above.m and stress_mpa.m.
assert(count_above([], 30.0) == 0);
assert(count_above([29.5 30.0 30.5], 30.0) == 1);
assert(abs(stress_mpa(10000, 500) - 20.0) < 1e-12);
% The complete run_checks.m also tests zero-area rejection.
```

Evidence labels

The MATLAB checks are supplied for a native run. Their expected values are independently derived; no local MATLAB execution is claimed.

[A] examples/matlab/run_checks.m; [W] Appendix I.28.

NAZARBAYEV UNIVERSITY

Plotting transfers as a sequence of questions

```python
time_s = np.linspace(0.0, 10.0, 101)
velocity_m_s = 9.81 * time_s
plt.plot(time_s, velocity_m_s)
plt.xlabel("Time (s)")
plt.ylabel("Velocity (m/s)")
plt.title("Constant-acceleration model")
plt.grid(True)
plt.show()
```

```octave
% A matching plotting example for the Python
    file plot_velocity.py.
time_s = linspace(0.0, 10.0, 101);
velocity_m_s = 9.81 * time_s;
plot(time_s, velocity_m_s);
xlabel('Time (s)');
ylabel('Velocity (m/s)');
title('Constant-acceleration model');
grid on;
```

Before interpreting

Which quantity is horizontal? Which is vertical? Are units, sample count, and model assumptions visible?

[W] Section 16.7.7; plotting commands adapted to 101 points.

NAZARBAYEV UNIVERSITY

Cooling: transfer the update rule

$$T_{i+1} = T_i - \Delta t\, k(T_i - T_{\text{ambient}}).$$

Initial temperature	$100\,^\circ\text{C}$
Ambient temperature	$25\,^\circ\text{C}$
Cooling constant	$k = 0.05\,\text{s}^{-1}$
Step and duration	$\Delta t = 1\,\text{s}$; 100 updates

A first check

$T_1 = 100 - 1(0.05)(100 - 25) = 96.25\,^\circ\text{C}.$

[W] Sections 16.8.1–16.8.4; Euler update supplied in the course book.

Python: append one new state per update

```python
# Core excerpt: the full function also validates its arguments.
def cooling_euler(n_steps=100, dt_s=1.0):
    times_s = [0.0]
    temperatures_c = [100.0]
    # Fixed model: ambient = 25 deg C; cooling constant = 0.05 /s.
    for step in range(n_steps):
        old_c = temperatures_c[-1]
        new_c = old_c - 0.05 * (old_c - 25.0) * dt_s
        times_s.append((step + 1) * dt_s)
        temperatures_c.append(new_c)
    return times_s, temperatures_c
```

Sample counting

Core excerpt from the supplied function: initialization plus 100 updates gives 101 time–temperature pairs.

[W] Section 16.8; [A] fixed-count adaptation with an explicit sample contract.

NAZARBAYEV UNIVERSITY

MATLAB: preallocate the same state sequence

```
% Store n_steps + 1 samples, including the initial condition.
times_s = (0:n_steps) * dt_s;
temperatures_c = zeros(size(times_s));
temperatures_c(1) = 100.0;
for i = 1:n_steps
    old_c = temperatures_c(i);
    % k = 0.05 /s and ambient temperature = 25 deg C.
    temperatures_c(i + 1) = old_c - 0.05 * (old_c - 25.0) * dt_s;
end
```

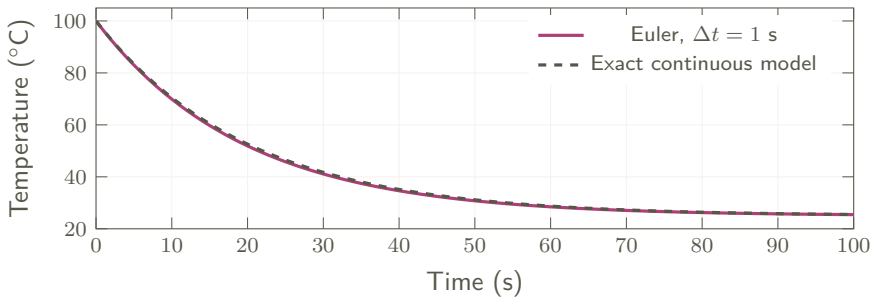
Map the physical sample

Python `temperatures_c[1]` and MATLAB `temperatures_c(2)` both refer to the state after the first update.

[W] Section 16.8.4; [A] explicit update-count adaptation.

NAZARBAYEV UNIVERSITY

A plot checks behavior, not the whole translation



Reference data generated in Python; not a measured cooling experiment or a MATLAB run.

[W] Appendix I.25; [A] data/cooling.csv from the included Python implementation.

Agreement and numerical accuracy are different checks

Test	What it establishes
101 samples; final t=100 s	The time-grid contract is satisfied
First update gives 96.25 deg C	The update has the intended sign and scale
$T_{100} = 25 + 75(0.95)^{100}$	The discrete recurrence is implemented
Compare with $25 + 75e^{-5}$	The Euler result differs from the continuous solution
Repeat with a smaller time step	A numerical-resolution check, not new sensor evidence

[W] Section 16.8 and Appendix I.25; original verification discussion.

NAZARBAYEV UNIVERSITY

Debugging clinic: a mixed-language translation

```
% INTENTIONALLY WRONG: identify the three translation defects.
time_s = [0 1 2 3];
height_m = 100 - 0.5 * 9.81 * time_s^2;
first_height = height_m(0);
print(first_height);
```

Pairs - repair the operations, index, and output - 4 min

Keep the intended model

Sample $h = 100 - \frac{1}{2}gt^2$ at the four specified times; then display the first height.

[W] Appendix I.26 and I.29.19; original shorter debugging adaptation.

NAZARBAYEV UNIVERSITY

A MATLAB translation checklist

1. Name the input and output quantities and units.
2. Check one-based positions and colon endpoints.
3. Inspect array sizes before using dotted operators.
4. Close each control block and function correctly.
5. Verify a known answer, a boundary, and one failure path.

Avoid a silent change

A translation that runs but selects different data or changes a rule is not equivalent.

[W] Appendix I.28 and I.31.

Lecture 38 exit ticket

Answer in words and short expressions

1. Translate Python `x[1:4]` for the same five-element vector.
2. Explain `A.^2` versus `A^2`.
3. Which MATLAB position stores the initial cooling temperature?
4. Why should the Euler result not equal the exact-model result exactly?

Individual - 3 min

[A] Lecture 38 synthesis.

NAZARBAYEV UNIVERSITY

LECTURE 39

C++ and Java: Read, Translate, and Test

Use explicit types without losing the engineering contract

NAZARBAYEV UNIVERSITY

Lecture 39 learning goals

1. Identify the core calculation inside a C++ or Java program.
2. Recognize declared types, blocks, loops, and callable units.
3. Distinguish value copying from reference aliasing.
4. Connect class state and validated updates to Week 9.
5. Reuse an independent test set across implementations.

[C] Session 13; [J1–J3]; original Java/C++ teaching additions.

NAZARBAYEV UNIVERSITY

C++: locate the familiar Ohm's-law calculation

```cpp
#include <iostream>  // Standard console output.
int main() {
    // Inputs are in volts and ohms; this example assumes R > 0.
    double voltage_v = 12.0;
    double resistance_ohm = 4.0;
    double current_a = voltage_v / resistance_ohm;
    std::cout << "Current = " << current_a << " A\n";
    return 0;  // Report successful execution to the operating system.
}
```

Read from the inside outward

The middle three numeric statements express the same engineering logic as the Python program.

[W] Section 16.9.1; original C++ version (the book prints a C example).

C++: build first, then run

```
# Terminal commands, not Python statements (compiler already installed).
g++ -std=c++17 -Wall -Wextra ohms_law.cpp -o ohms_law
./ohms_law
```

Windows PowerShell variant

With a Windows g++ toolchain, build `ohms_law.exe` and run `.\ohms _law.exe`.

Separate two diagnoses

A compiler message is different from an error raised while a built program is running.

[A] Commands executed with g++ using C++17; compiler documentation in source notes.

NAZARBAYEV UNIVERSITY

Java: find the same calculation inside main

```java
public class OhmsLaw {
    public static void main(String[] args) {
        // Inputs are in volts and ohms; this example assumes R > 0.
        double voltageV = 12.0;
        double resistanceOhm = 4.0;
        double currentA = voltageV / resistanceOhm;
        System.out.println("Current = " + currentA + " A");
    }
}
```

The file is `OhmsLaw.java`

The public class name and filename match in this ordinary example.

[J1] Getting Started with Java; original engineering example.

NAZARBAYEV UNIVERSITY

Java: source, class files, and the JVM

```
# Terminal commands in the folder containing OhmsLaw.java.
javac OhmsLaw.java
java OhmsLaw
```

Term in this example	Purpose
javac	Compile source to class files
java	Launch the Java runtime for the class
static main	An entry method called without making an OhmsLaw instance
void	This method does not return a value

[J1] Java compilation and execution; [J2] language basics.

NAZARBAYEV UNIVERSITY

Declared types do not settle numeric meaning

```
// Both C++ and Java: operands determine
     division.
int samples = 7;
int groups = 2;
double wrong = samples / groups;
// Convert an operand before dividing.
double correct = 1.0 * samples / groups;
```

Expected values

- wrong is 3.0.
- correct is 3.5.
- Both destinations have a floating type.

Range also matters

C++ and Java integer types have finite ranges. Python integers have different range behavior.

[C1] expr.mul; [J2] primitive types; [P] numeric types.

NAZARBAYEV UNIVERSITY

Do not copy a chained comparison literally

```python
# Python supports this chained comparison.
within = 20.0 <= temperature_c <= 30.0
```

```java
// C++ and Java: compare the quantity twice.
boolean within = temperatureC >= 20.0
    && temperatureC <= 30.0; // Java
// Use bool instead of boolean in C++.
```

A dangerous translation

Java rejects the numeric chained comparison. In C++, a comparison result can be compared again and produce a misleading answer.

[J3] JLS 15.20 and15.23; [C2] C++ relational expressions; original comparison.

NAZARBAYEV UNIVERSITY

New loop syntax, the same repeated task

```python
# Python: visit each reading in order.
for value in values:
    if value > limit:
        count += 1
```

```cpp
// C++ range-for header:
// for (double value : values) { ... }
// Java enhanced-for header is the same here.
for (double value : values) {
    if (value > limit) {
        ++count;
    }
}
```

Trace the boundary

For values 29.5, 30.0, 30.5 and limit 30.0, only the last entry increments the count.

[J2] control flow; [A] matched count implementation; Weeks 2 and10.

NAZARBAYEV UNIVERSITY

C++ function: explicit inputs and result type

```cpp
inline int count_above(const std::vector<double>& values, double limit) {
    // Read without copying or modifying the vector; assume finite values.
    int count = 0;
    for (double value : values) {
        if (value > limit) {  // Strictly above; equality is excluded.
            ++count;
        }
    }
    return count;
}
```

Read the interface: `int` is the result type; `double limit` is the threshold.

`const std::vector<double>&` reads the vector without copying it here.

[C3] C++ vector interface; [A] week13_models.hpp.

NAZARBAYEV UNIVERSITY

Java method: the same strict-count contract

```java
public static int countAbove(double[] values, double limit) {
    // Read each array entry; assume a non-null array of finite values.
    int count = 0;
    for (double value : values) {
        if (value > limit) {  // Strict comparison matches Python.
            ++count;
        }
    }
    return count;
}
```

Call through its class

`Week13Models.countAbove(values, 30.0)` uses a static method; no model instance is required.

[J2] arrays and methods; original Week13Models implementation.

NAZARBAYEV UNIVERSITY

Choose the corresponding collection deliberately

Collection	First / last	Length / size
Python list	`x[0]` / `x[-1]`	`len(x)`
C++ vector	`x[0]` / `x.back()`	`x.size()`
Java array	`x[0]` / `x[x.length-1]`	`x.length`
MATLAB vector	`x(1)` / `x(end)`	`numel(x)`

Nonempty precondition

A request for the last element needs at least one element. Index checks and failure behavior differ by interface.

[C3] vector; [J2] arrays; [M1] indexing; [P] sequences.

NAZARBAYEV UNIVERSITY

String content is not always tested with ==

```cpp
// std::string equality compares text content
    , not address identity.
std::string label = "kPa";
std::cout << std::boolalpha << (label == "kPa
    ") << '\n';
```

```java
// Construct a distinct object to expose
    reference comparison.
String label = new String("kPa");
System.out.println(label == "kPa");       //
    false: identity.
System.out.println(label.equals("kPa"));  //
    true: text content.
```

State what you are comparing

Use `std::string` content equality in C++; use `equals()` for Java string content.

[J3] JLS15.21.3; [C4] basic_string comparison; original verified examples.

NAZARBAYEV UNIVERSITY

The same assignment spelling can copy different things

```cpp
// Ordinary vector copy owns a separate
    sequence of numeric values.
std::vector<double> a = {10.0, 20.0};
std::vector<double> b = a;
b[0] = 99.0;
std::cout << a[0] << '\n';  // 10, unlike a
    Python list alias.
```

```java
// Assignment copies an array reference, not
    its elements.
double[] a = {10.0, 20.0};
double[] b = a;
b[0] = 99.0;
System.out.println(a[0]);  // 99.0, as with a
     Python list alias.
```

Expected original first value

C++ numeric vector: 10. Java array: 99. A Python list alias behaves like the latter in this example.

[C3] vector; [J2] arrays; [P] names and objects; executed examples.

NAZARBAYEV UNIVERSITY

Python baseline: query and update state

```
class Sensor:
    """A small state model: accept only finite nonnegative readings."""
    def __init__(self, value):
        self.update(value)
    def update(self, value):
        candidate = finite_number(value)
        if candidate < 0.0:
            raise ValueError("reading must be nonnegative")
        self._value = candidate  # Commit only after validation.
    def get_value(self):
        return self._value
```

Keep the promise

Accept only finite, nonnegative readings. A rejected update leaves the earlier reading intact.

[A] Original bridge from Weeks 8–9; engineering_models.py.

NAZARBAYEV UNIVERSITY

C++: constructor, methods, and private state

```
class Sensor {
private:
    double value_ = 0.0;  // One stored reading per object.
public:
    explicit Sensor(double value) { update(value); }
    void update(double value) {
        if (!std::isfinite(value) || value < 0.0) {
            throw std::invalid_argument("invalid reading");
        }
        value_ = value;  // Commit after validation.
    }
    double get_value() const { return value_; }
};
```

Roles: constructor initializes; `update` changes state; `get_value` queries. `private` restricts direct source-level access.

[C2] class member access; original Sensor model.

Java: the same state-change contract

```
public static class Sensor {
    private double value;  // Per-instance state, not a static field.
    public Sensor(double value) { update(value); }
    public void update(double value) {
        if (!Double.isFinite(value) || value < 0.0) {
            throw new IllegalArgumentException("invalid reading");
        }
        this.value = value;  // Commit after validation.
    }
    public double getValue() { return value; }
}
```

Read `this.value = value`

The field on this object receives the local parameter only after validation succeeds.

[J2] classes and objects; original nested Sensor model.

NAZARBAYEV UNIVERSITY

Relate new syntax to the Week 9 ideas

Concept	Python	C++	Java
Receiver	`self`	`this` pointer	`this` reference
Initializer / constructor	`__init__`	Class-name constructor	Class-name constructor
Internal-state convention	`_value`	`private:`	`private` field
Inheritance notation	`class B(A):`	`class B : public A`	`class B extends A`

Do not infer identical semantics

Naming conventions, access checks, dispatch, lifetime, and copying still need the target language's rules.

[W] Week9 concepts; [P] classes; [J2] classes; [C2] classes.

NAZARBAYEV UNIVERSITY

A rejected update must preserve the last valid state

```
# Same behavior is tested in all three
    implementations.
sensor = Sensor(10.0)
sensor.update(12.0)
try:
    sensor.update(-1.0)
except ValueError:
    print("Rejected")
print(sensor.get_value())  # Must still be
    12.0.
```

Target-language exception names

- C++: `std::invalid_argument`.
- Java: `IllegalArgumentException`.
- The exception spelling changes; the state promise does not.

[A] Python, C++, and Java independent state-preservation tests.

NAZARBAYEV UNIVERSITY

Use one small, independently specified test set

Case	Independent expectation
10000 N; 500 mm^2	20 MPa
0 N; positive area	0 MPa
Zero area	Reject before division
29.5, 30.0, 30.5; limit30	One strictly high value
Update 10 to12, then propose -1	Reject -1 and retain12

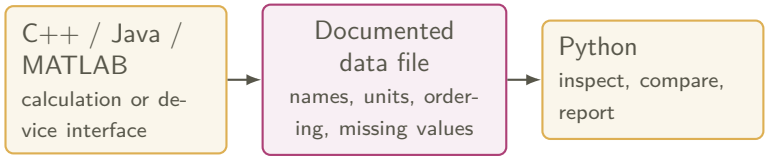
Executed evidence

Python: 20 unit-test methods. C++: 27 checks. Java: 29 checks. Fifteen shared-output comparisons passed.

[A] tests/verify_all.py and data/verification.json.

NAZARBAYEV UNIVERSITY

Exchange data without sharing source syntax



A small schema

`time_s,temperature_c,status`

Agree on decimal notation, column meanings, sample order, and failure indicators.

[W] Sections 16.5,16.9; original interoperability illustration.

NAZARBAYEV UNIVERSITY

Activity: translate the first-threshold search

```
# The result is the first position, not the count of matches.
def first_above(values, limit):
    for index, value in enumerate(values):
        if value > limit:
            return index
    return None
```

Propose a C++ or Java interface

1. Choose an absence result explicitly.
2. Write first-position and absent-target tests.
3. Explain how the caller distinguishes position zero from absence.

Pairs - pseudocode or short code - 5 min

[A] Original transfer exercise using Week12 first-match contract.

NAZARBAYEV UNIVERSITY

Review a translation like an engineering change

1. Compare input meanings, dimensions, and ordering.
2. Check strict versus inclusive comparisons.
3. Inspect copying, mutation, and exception behavior.
4. Repeat the original known-answer and boundary tests.
5. Record the toolchain and the changes you accepted.

A key question

What could this program get wrong even though it compiles successfully?

[W] Sections 16.6,16.11,16.13; original synthesis.

NAZARBAYEV UNIVERSITY

Use the tool that makes the evidence strongest

Python remains valuable

- Orchestrate tasks and inspect results.
- Keep a readable reference implementation.
- Reuse libraries where appropriate.

Another language may fit another constraint

- Existing codebase or hardware interface.
- Required library or numerical environment.
- Measured performance and maintenance needs.

Avoid a false trade-off

Language choice does not replace model choice, algorithm choice, or verification.

[W] Sections 16.9–16.11; Weeks10–12 connection.

NAZARBAYEV UNIVERSITY

Lecture 39 exit ticket: what travels with you?

Three answers

1. Name one translation defect that a compiler may not detect.
2. Explain why Java array assignment and C++ vector copying differ here.
3. Name two tests you would retain when porting an engineering function.

Individual - 3 min

Week 14

Review the complete course through problems, algorithms, trustworthy code, and engineering interpretation.

[C] Session 14; [A] Lecture39 synthesis.

Optional: solve the same matrix equations

```python
import numpy as np
A = np.array([[3.0, 2.0], [1.0, 4.0]])
b = np.array([12.0, 10.0])
x = np.linalg.solve(A, b)  # Solve; do not
    explicitly invert A.
residual = A @ x - b
print("solution:", x.tolist())
print(f"residual norm: {np.linalg.norm(
    residual):.3e}")
```

```octave
% Optional example from the course book;
    solve A*x = b.
A = [3.0 2.0; 1.0 4.0];
b = [12.0; 10.0];            % A column vector
    .
x = A \ b;                  % Solve without
    computing inv(A).
residual = A * x - b;
disp(x);
fprintf('residual norm: %.3e\n', norm(
    residual));
```

Independent answer

$x_1 = 2.8$, $x_2 = 1.8$; substitute into both equations and inspect the residual.

[W] Section 16.7.11 and Appendix I.16; optional library example.

Optional: SQL expresses a requested result

```sql
-- Select high-temperature rows in time order.
SELECT time_s, temperature_c
FROM readings
WHERE temperature_c > 80
ORDER BY time_s;
```

Python can coordinate the query

The included `sql_bridge.py` creates a small in-memory database and obtains `[(1.0, 85.0), (2.0, 82.0)]`.

Different interface, familiar question

Filter rows and preserve a deliberate output order.

[W] Section 16.9.2; [A] executed sqlite3 teaching example.

NAZARBAYEV UNIVERSITY

Optional: C is a different language from C++

```c
/* Optional comparison: C, not C++. The inputs assume positive resistance. */
#include <stdio.h>
int main(void) {
    double voltage_v = 12.0;       /* V */
    double resistance_ohm = 4.0;  /* ohm */
    double current_a = voltage_v / resistance_ohm;
    printf("Current = %.2f A\n", current_a);
    return 0;
}
```

Why show it?

The course book's first C/C++ discussion uses this C-style structure. The main lecture develops C++ separately.

[W] Section 16.9.1; explicit C adaptation; compiled as C11.

NAZARBAYEV UNIVERSITY

Optional: other tools to recognize

Tool	A possible engineering role
SQL	Queries over structured stored records
JavaScript	Browser interfaces and interactive dashboards
R	Statistical analysis and reporting
Julia	Scientific and numerical programming
Shell tools	Automating file and command workflows
LabVIEW	Graphical instrumentation and data acquisition

Awareness, not an installation list

Learn another environment when the task, team, or course requires it.

[W] Sections 16.9.2–16.10.

NAZARBAYEV UNIVERSITY

Quick reference: preserve meaning across syntax

Idea	Python	MATLAB	C++ / Java
First element	`x[0]`	`x(1)`	`x[0]`
Square a scalar	`x**2`	`x^2`	`x*x`
Scalar AND	`and`	`&&`	`&&`
Block ending	dedent	`end`	`}`
Comment	`#`	`%`	`//`
Numeric approximate equality	`abs(a-b)<=tol`	`abs(a-b)<=tol`	appropriate abs function

[P], [M1–M5], [J2–J3], [C1–C3]; reference synthesis.

NAZARBAYEV UNIVERSITY

How to use the accompanying example package

- Run small examples from the appropriate language folder.
- Read the README for exact commands and required tools.
- Use `tests/verify_all.py` to rerun available Python, C++, and Java checks.
- In MATLAB, run `examples/matlab/run_checks.m` from its folder.

Verification boundary

Python, C++, Java, and the optional C example were executed locally. MATLAB listings were reviewed against official documentation, but not executed in MATLAB.

[A] README.md; data/verification.json.

NAZARBAYEV UNIVERSITY

Sources: course, Python, and MATLAB

- **[C]** Nazarbayev University, *ENG 101 Course Specification Report*, Fall 2026, Session 13; Session 14 for the next scope boundary.
- **[W]** Y. Wang, *Programming for Engineers with Python*, supplied manuscript v20260815, Sections 16.5–16.13; selected Appendix I.
- **[P]** Python Software Foundation, Python Tutorial: numbers, modules, classes, and floating-point behavior.
- **[M]** MathWorks documentation: array indexing; array/matrix operations; colon, strcmp, and function declarations.

Adaptation policy

Source scope is retained. Added comparisons, tests, and examples are labelled as teaching adaptations; the full source map is included.

[A] Full references with direct links are recorded in SOURCE_MAP.md.

NAZARBAYEV UNIVERSITY

Source basis: C++, Java, and verification

- **[J1–J2]** Oracle/OpenJDK learning materials at dev.java: execution, language basics, arrays, and classes.
- **[J3]** Java Language Specification, Chapter 15: operators and reference equality.
- **[C1–C4]** Public C++ working draft: division, expressions, vector, and string comparison.
- **[N]** NumPy documentation: arrays, elementwise operations, and `linspace`.

No universal tool ranking

Examples teach transferable reasoning and documented semantics, not claims that one language is always fastest or best.

[A] References checked for this package; no performance benchmark is claimed.

Week 13: keep the reasoning, learn the tool

A language changes how you express a solution.

It does not remove your responsibility for the model, units, data, algorithm, tests, and interpretation.

Next: Week 14 course review